

iOS SDK Development Guide

[TOC]

Development Guide

1 Environment configuration

1.1 Add framework

Add QCBandSDK.framework to the project, the framework supports iOS 9.0 and above

Note:Because the classification is used in the framework, you need to add settings to the project

Target->Build Settings -> Other Linker Flags add **-ObjC**

Add the following code in Target-Build Settings-Excluded Architectures

```
EXCLUDED_ARCHS__EFFECTIVE_PLATFORM_SUFFIX_simulator__NATIVE_ARCH_64_BIT_x86_64=arm64 arm64e armv7 armv7s armv6 armv8
```



1.2 Configure Bluetooth permissions

Configure bluetooth permissions in info.plist file

```
<key>NSBluetoothAlwaysUsageDescription</key>
<string>App needs to use your bluetooth device</string>
<key>NSBluetoothPeripheralUsageDescription</key>
<string>App needs to use your bluetooth device</string>
```

2 Usage

2.1 Service UUIDs supported by the device

Defined in QCSDKManager.h, the service UUID supported by the device:

```
extern NSString *const QCBANDSDKSERVERUUID1;
extern NSString *const QCBANDSDKSERVERUUID2;
```

2.2 import framework library

Introduce the framework library into the code

```
#import <QCBandSDK/QCSDKManager.h>
#import <QCBandSDK/QCSDKCmdCreator.h>
```

Initialize [QCSDKManager sharedInstance] with a singleton

```
QCSDKManager:Peripherals for joining connections
QCSDKCmdCreator:Used to send commands to peripherals
```

2.3 Scan Ring

initialization

Scanning can only be started when permissions are allowed and Bluetooth is turned on.

Import Apple's CoreBluetooth library and follow two protocols <CBCentralManagerDelegate, CBPeripheralDelegate>

```
#import <CoreBluetooth/CoreBluetooth.h>
```

Declare central and peripheral roles

```

/*Central Role,app*/
@property (strong, nonatomic) CBCentralManager *centerManager;

/*Peripheral role, scanned peripherals*/
@property (strong, nonatomic) NSMutableArray<CBPeripheral *> *peripherals;

/*Connected peripheral role*/
@property (strong, nonatomic) CBPeripheral *connectedPeripheral;

```

Instantiate the central role

```

self.centerManager = [[CBCentralManager alloc] initWithDelegate:self queue:nil];

```

Scan Ring

Using Scan Peripherals

```

NSArray *serviceUUIDStrings = @[QCBANDSDKSERVERUUID1,QCBANDSDKSERVERUUID2];

NSMutableArray *uuids = [NSMutableArray array];
for (id obj in serviceUUIDStrings) {
    if ([obj isKindOfClass:[NSString class]]) {
        CBUUID *uuid = [CBUUID UUIDWithString:obj];
        [uuids addObject:uuid];
    }
}

NSDictionary *option = @{CBCentralManagerScanOptionAllowDuplicatesKey : [NSNumber numberWithInt:NO]};
[self.centerManager scanForPeripheralsWithServices:uuids options:option];

```

Note: To obtain the scanned peripheral devices in the agent, you can perform secondary filtering through the device name and other related information.

```

- (void)centralManager:(CBCentralManager *)central didDiscoverPeripheral:(CBPeripheral *)peripheral
  advertisementData:(NSDictionary<NSString *,id> *)advertisementData RSSI:(NSNumber *)RSSI {
    if (peripheral.name.length > 0) {
        [self.peripherals addObject:peripheral];
        [self.deviceList reloadData];
    }
}

```

2.4 Cancel the scan of the Ring

Call the interface of the central role to stop scanning

```

[self.centerManager stopScan];

```

2.5 Connect the Ring

start connecting

```

self.connectedPeripheral = self.peripherals[indexPath.row];
[self.centerManager connectPeripheral:self.connectedPeripheral options:nil];

```

After the connection is successful, pass in the peripheral device to the SDK

```

- (void)centralManager:(CBCentralManager *)central didConnectPeripheral:(CBPeripheral *)peripheral {
    [[QCSDKManager sharedInstance] addPeripheral:peripheral];
}

```

2.6 Disconnect

```

[self.centerManager cancelPeripheralConnection:self.connectedPeripheral];

```

After disconnecting, remove peripherals

```

- (void)centralManager:(CBCentralManager *)central didDisconnectPeripheral:(CBPeripheral *)peripheral error:
  (nullable NSError *)error {
    [[QCSDKManager sharedInstance] removePeripheral:peripheral];
}

```

3. Instructions supported by the device

3.1 Set Ring time

```
/**
 * Set the time of the Ring
 */
+ (void)setTime:(NSDate *)date success:(void (^)(NSDictionary *))suc failed:(void (^)(void))fail;
```

3.2 Read Ring battery

```
/*!
 * @func Read Ring battery
 * @param suc battery:Power level(0~8)
 */
+ (void)readBatterySuccess:(void (^)(int battery))suc failed:(void (^)(void))fail;
```

3.3 Bound Vibration

```
/**
 * Bound Vibration
 */
+ (void>alertBindingSuccess:(nullable void (^)(void))suc fail:(nullable void (^)(void))fail;
```

3.4 Set wristband time base/user personal information

```
/**
 * Set wristband time base/user personal information

@param twentyfourHourFormat : YES 24 hour system; NO 12 hour system
@param metricSystem        : YES Metric; NO Imperial
@param gender               : gender (0=male, 1=female)
@param age                  : age (years)
@param height               : height (cm)
@param weight               : weight (kg)
@param sbpBase              : systolic blood pressure base (mmhg) (reserved value, Defaults:0)
@param dbpBase              : Diastolic blood pressure base (mmhg) (reserved value, Defaults:0)
@param hrAlarmValue         : Heart rate alarm value (bpm) (reserved value, Defaults:0)
 *
 *
 */
+ (void)setTimeFormatTwentyfourHourFormat:(BOOL)twentyfourHourFormat
metricSystem:(BOOL)metricSystem
gender:(NSInteger)gender
age:(NSInteger)age
height:(NSInteger)height
weight:(NSInteger)weight
sbpBase:(NSInteger)sbpBase
dbpBase:(NSInteger)dbpBase
hrAlarmValue:(NSInteger)hrAlarmValue
success:(void (^)(BOOL, BOOL, NSInteger, NSInteger, NSInteger, NSInteger, NSInteger,
    NSInteger))success
fail:(void (^)(void))fail;
```

3.5 Get Ring time base/user personal information

```
/**
 * Get Ring time base/user personal information

@param success isTwentyfour: YES 24 hour system; NO 12 hour system
               isMetricSystem: YES Metric; NO Imperial
               gender: gender (0=male, 1=female)
               age: age (years)
               height: height (cm)
               weight: weight (kg)
               sbpBase: systolic blood pressure base (mmhg) (reserved value, Defaults:0)
               dbpBase: Diastolic blood pressure base (mmhg) (reserved value, Defaults:0)
               hrAlarmValue: Heart rate alarm value (bpm) (reserved value, Defaults:0)
 *
 */
+ (void)getTimeFormatInfo:(nullable void (^)(BOOL isTwentyfour, BOOL isMetricSystem, NSInteger gender, NSInteger
    age, NSInteger height, NSInteger weight, NSInteger sbpBase, NSInteger dbpBase, NSInteger
    hrAlarmValue))success fail:(nullable void (^)(void))fail;
```

3.6 Get the version number of the Ring firmware

```
/**
 * @func Get the version number of the Ring firmware
 *
 * @param success The format of software and hardware version numbers is generally"x.x.x"
 */
+ (void)getDeviceSoftAndHardVersionSuccess:(void (^)(NSString *_Nonnull, NSString *_Nonnull))success fail:(void
    (^)(void))fail;
```

3.7 Get current steps

```
/*!
 * @func Get current steps
 */
+ (void)getCurrentSportSuccess:(void (^)(SportModel *sport))suc failed:(void (^)(void))fail;
```

3.8 Get total statistics for a day(Steps、Calories、Distance、Time)

```
/*!
 * @func Get total statistics for a day
 *
 * @param index : 0->Today 1->1 day ago.Maximum 29
 * @param suc :Using this command cannot accurately obtain the statistical data of the day, the device will
    save the data every 15 minutes, so there will be a 15-minute interval
 */
+ (void)getOneDaySportBy:(NSInteger)index success:(void (^)(SportModel *model))suc fail:(void (^)(void))fail;
```

3.9 Get detailed exercise data for a day

```
/*!
 * @func Get detailed exercise data for a day
 * @discussion There is a tick every 15 minutes, and there will be a maximum of 96 pieces of data per day. For
    details, please see the returned content
 * @param items sports:return all sports models
 */
+ (void)getSportDetailDataByDay:(NSInteger)dayIndex sportDatas:(nullable void (^)(NSArray<SportModel *>
    *sports))items fail:(nullable void (^)(void))fail;
```

3.10 Get detailed exercise data for a specified time period on a certain day

```
/*!
 * @func Get detailed exercise data for a specified time period on a certain day
 * @param minuteInterval minute interval for each index
 * @param beginIndex time period start index
 * @param endIndex time period end index
 * @param items sports:return all sports models
 */
+ (void)getSportDetailDataByDay:(NSInteger)dayIndex minuteInterval:(NSInteger)minuteInterval beginIndex:
    (NSInteger)beginIndex endIndex:(NSInteger)endIndex sportDatas:(nullable void (^)(NSArray<SportModel *>
    *sports))items fail:(nullable void (^)(void))fail;
```

Demo: Today's activity score

In the Demo, open **Health Settings → Activity & Steps → Today's Steps & Activity Score**. The row prompts for age, obtains today's total steps with `getCurrentSportSuccess: failed:`, and calculates an activity score from 0 to 100.

The score is Demo-side business logic, not a device protocol value or a standalone QCSDKCmdCreator API. QCSampleHealthScoreUtils uses these age-based step divisors:

- Ages 1–12: 150
- Ages 13–18: 160
- Ages 19–35: 187
- Ages 36–59: 125
- Ages 60 and above: 100

The calculation is `MIN(todaySteps / ageDivisor, 100)`. Display levels are: 0–59 "Keep Moving," 60–74 "Fair," 75–84 "Good," and 85–100 "Excellent."

3.11 Get sleep data for a day

Sleep data is assigned to the calendar day on which the sleep ends. Overnight sleep is calculated backwards from its end time, so the API supports full-day and cross-midnight data.

```
typedef NS_ENUM(NSInteger, SLEEPTYPE) {
    SLEEPTYPENONE = 0, // No data
    SLEEPTYPESOBER, // Awake
    SLEEPTYPELIGHT, // Light sleep
    SLEEPTYPEDEEP, // Deep sleep
    SLEEPTYPEREM, // REM
    SLEEPTYPEUNWEARED // Not worn
};

typedef NS_ENUM(NSInteger, QCSleepDetailDataType) {
    QCSleepDetailDataTypeUnknown = 0,
    QCSleepDetailDataTypeMainSleepBegin = 1,
    QCSleepDetailDataTypeMainSleepEnd = 2,
    QCSleepDetailDataTypeOtherSleepBegin = 3,
    QCSleepDetailDataTypeOtherSleepEnd = 4,
    QCSleepDetailDataTypeSleeping = 5
};

@interface QCSleepModel : NSObject
@property (nonatomic, assign) SLEEPTYPE type;
@property (nonatomic, strong) NSString *happenDate; // yyyy-MM-dd HH:mm:ss
@property (nonatomic, strong) NSString *endTime; // yyyy-MM-dd HH:mm:ss
@property (nonatomic, assign) NSInteger total; // Duration in minutes
@property (nonatomic, assign) BOOL isMidday; // YES for nap data
@property (nonatomic, assign) NSInteger effectiveMinutes;
@property (nonatomic, copy) NSString *sleepQa; // Raw quality for indexed sleep details
@property (nonatomic, assign) QCSleepDetailDataType dataType;
@end

// Main sleep for the specified day.
+ (void)getSleepDetailDataByDay:(NSInteger)dayIndex
    sleepDatas:(nullable void (^)(NSArray<QCSleepModel *> *sleeps))items
    fail:(nullable void (^)(void))fail;

// Main sleep and naps for the specified day.
+ (void)getFullDaySleepDetailDataByDay:(NSInteger)dayIndex
    sleepDatas:(nullable void (^)(NSArray<QCSleepModel *> *sleeps,
        NSArray<QCSleepModel *> *naps))items
    fail:(nullable void (^)(void))fail;

// Indexed sleep detail query. A 15-minute interval uses indexes 0...95 for a full day.
+ (void)getSleepDetailDataByDay:(NSInteger)dayIndex
    minuteInterval:(NSInteger)minuteInterval
    beginIndex:(NSInteger)beginIndex
    endIndex:(NSInteger)endIndex
    sleepDatas:(nullable void (^)(NSArray<QCSleepModel *> *sleeps))items
    fail:(nullable void (^)(void))fail;
```

dayIndex is relative to today: 0 means today and 1 means yesterday. Nap-capable devices return nap data separately from main sleep data. An empty array is a valid no-data result.

Demo: Today's sleep score and efficiency

In the Demo, open **Health Settings → Sleep → Today's Sleep Score & Efficiency**. This row always queries dayIndex = 0 and analyzes today's main sleep only; naps are excluded.

QCSampleHealthScoreUtils aggregates deep, light, REM, and awake minutes before calculating a sleep score from 0 to 100. Sleep efficiency is a UI mapping of that score, not an independent physiological efficiency algorithm:

- Score below 60: 80%
- 60–69: 82%
- 70–79: 85%
- 80–84: 90%
- 85–89: 95%
- 90–94: 99%
- 95–100: 100%

Both values are Demo-side display logic rather than fields returned directly by the device. When no valid main-sleep segments are available, the Demo reports that no valid sleep data exists.

For protocol layout, state mapping, cross-day calculations, and integration checks, see [Sleep Data Synchronization and Detail Integration Guide](#).

3.12 Find Ring

```
/**
 * Find Ring
 */
+ (void)lookupDeviceSuccess:(void (^)(void))suc fail:(void (^)(void))fail;
```

3.13 Ring to the camera interface

```
/**
 * Switch the lower computer to the camera interface
 */
+ (void)switchToPhotoUISuccess:(nullable void (^)(void))success fail:(nullable void (^)(void))fail;
```

3.14 keep the camera interface

```
/**
 * Keep the camera interface of the lower computer
 */
+ (void)holdPhotoUISuccess:(nullable void (^)(void))success fail:(nullable void (^)(void))fail;
```

3.15 Stop taking pictures

```
/**
 * Stop the lower computer to take pictures
 */
+ (void)stopTakingPhotoSuccess:(nullable void (^)(void))success fail:(nullable void (^)(void))fail;
```

3.16 Restart the Ring

```
/**
 * Restart the Ring
 */
+ (void)resetBandHardlySuccess:(nullable void (^)(void))suc fail:(nullable void (^)(void))fail;
```

3.17 Get Ring Mac address

```
/**
 * @func Get Ring Mac address
 * @param success The Mac address format is "AA:BB:CC:DD:EE:FF"
 */
+ (void)getDeviceMacAddressSuccess:(nullable void (^)(NSString *_Nullable macAddress))success fail:(nullable void (^)(void))fail;
```

3.18 Get information about the timed blood pressure measurement function

```
/**
 * Get information about the timed blood pressure measurement function
 * @param success featureOn YES: ON; NO: OFF
 * beginTime Start time, format "HH:mm"
 * endTime End time, the format is "HH:mm"
 * minuteInterval minute interval (minutes)
 */
+ (void)getSchedualBPInfo:(nullable void (^)(BOOL featureOn, NSString *beginTime, NSString *endTime, NSInteger minuteInterval))success fail:(void (^)(void))fail;
```

3.19 Information on setting the timed blood pressure measurement function

```
/**
 * Information on setting the timed blood pressure measurement function
 * @param featureOn YES: ON; NO: OFF
 * @param beginTime Start time, format "HH:mm"
 * @param endTime End time, the format is "HH:mm"
 * @param minuteInterval minute interval (minutes)
 */
+ (void)setSchedualBPInfoOn:(BOOL)featureOn beginTime:(NSString *)beginTime endTime:(NSString *)endTime
    minuteInterval:(NSInteger)minuteInterval success:(nullable void (^)(BOOL featureOn, NSString *beginTime,
    NSString *endTime, NSInteger minuteInterval))success fail:(void (^)(void))fail;
```

3.20 Obtain historical data for timed blood pressure measurements

```
/**
 * Obtain historical data for timed blood pressure measurements
 * @param userAge :User age
 * @param success data: Heart rate module data, the current reply is actually unified as heart rate, which can
    be processed by itself in the callback
 */
+ (void)getSchedualBPHistoryDataWithUserAge:(NSInteger)userAge success:(nullable void (^)(
    NSArray<BloodPressureModel *> *data))success fail:(nullable void (^)(void))fail;
```

3.21 Reset the Ring to factory settings

```
/**
 * Reset the Ring to factory settings
 */
+ (void)resetBandToFacotrySuccess:(nullable void (^)(void))success fail:(nullable void (^)(void))fail;
```

3.22 Get historical data of exercise records

```
/**
 * @func Get historical data of exercise records
 * @param lastUnixSeconds The time when the last exercise data occurred (seconds since 1970-01-01 00:00:00)
 * @note success models Motion data array
 */
+ (void)getExerciseDataWithLastUnixSeconds:(NSInteger)lastUnixSeconds getData:(nullable void (^)(
    NSArray<ExerciseModel *> *models))getData fail:(nullable void (^)(void))fail;
```

3.23 Get historical data for manual blood pressure measurements

```
/**
 * Get historical data for manual blood pressure measurements
 * @param lastUnixSeconds Time when the last manual blood pressure data occurred (seconds since 1970-01-01
    00:00:00)
 * @param success data blood pressure data array
 */
+ (void)getManualBloodPressureDataWithLastUnixSeconds:(NSInteger)lastUnixSeconds success:(nullable void (^)(
    NSArray<BloodPressureModel *> *data))success fail:(nullable void (^)(void))fail;
```

3.24 Get timed heart rate historical data

```
/**
 * @func Get timed heart rate historical data
 * @param dates: List of dates for which historical data needs to be obtained
 * @note success models Timed heart rate data array
 */
+ (void)getSchedualHeartRateDataWithDates:(NSArray<NSDate *> *)dates success:(nullable void (^)(
    NSArray<SchedualHeartRateModel *> *models))success fail:(nullable void (^)(void))fail;

/**
 * @func Get timed heart rate historical data
 * @param dayIndexs The number of days for which historical data needs to be obtained (0->today, 1->yesterday, 2-
    >the day before yesterday, and so on)
 * @note success models Timed heart rate data array
 */
+ (void)getSchedualHeartRateDataWithDayIndexs:(NSArray<NSNumber*> *)dayIndexs success:(void (^)(
    NSArray<QCSchedualHeartRateModel *> *_Nonnull))success fail:(void (^)(void))fail;
```

3.25 Get information about the timed heart rate function

```
/**
 * Get information about the timed heart rate function
 * @param success enable Whether the timed heart rate function is enabled. YES: ON; NO: OFF
 */
+ (void)getSchedualHeartRateStatusWithCurrentState:(BOOL)enable success:(nullable void (^)(BOOL enable))success
  fail:(nullable void (^)(void))fail;
```

3.26 Information on setting the timed heart rate function

```
/**
 * Information on setting the timed heart rate function
 * @param enable Whether the timed heart rate function is enabled. YES: ON; NO: OFF
 */
+ (void)setSchedualHeartRateStatus:(BOOL)enable success:(nullable void (^)(BOOL enable))success fail:(nullable
  void (^)(void))fail;
```

3.27 According to the specified time stamp, the new version of Sports+ (V2) data summary information

```
/**
 According to the specified time stamp, the new version of Sports+ (V2) data summary information
 @param timestamp
 @param finished spSummary - Motion+Summary info array
 */
+ (void)getSportPlusSummaryFromTimestamp:(NSTimeInterval)timestamp finished:(nullable void (^)(NSArray *_Nullable
  spSummary, NSError *_Nullable error))finished;
```

3.28 According to the specified new version of the campaign + summary information, get some summary information and detailed data of the campaign

```
/**
 According to the specified new version of the campaign + summary information, get some summary information and
 detailed data of the campaign
 @param finished spSummary - Motion+Summary info array
 */
+ (void)getSportPlusDetailsWithSummary:(OdmGeneralExerciseSummaryModel *)summary finished:(nullable void (^)(
  OdmGeneralExerciseSummaryModel *_Nullable summary, OdmGeneralExerciseDetailModel *_Nullable detail,
  NSError *_Nullable error))finished;
```

3.29 Get/set user target information

```
/**
 Get/set user target information

 * @param suc stepTarget:      Step target
  calorieTarget:      Calorie Goal, Unit: Calories
  distanceTarget:      Distance to target, unit: meters
  sportDuration:      Exercise duration target Unit: minutes (reserved value, default: 0)
  sleepDuration:      Sleep duration target unit: minutes (reserved value, default: 0)
 */
+ (void)getStepTargetInfoWithSuccess:(nullable void (^)(NSInteger stepTarget, NSInteger calorieTarget, NSInteger
  distanceTarget, NSInteger sportDuration, NSInteger sleepDuration))suc fail:(nullable void (^)(void))fail;

/**
 Set user target information

 * @param stepTarget:      Step target
 * @param calorieTarget:      Calorie Goal, Unit: Calories
 * @param distanceTarget:      Distance to target, unit: meters
 * @param sportDuration:      Exercise duration target Unit: minutes (reserved value, default: 0)
 * @param sleepDuration:      Sleep duration target unit: minutes (reserved value, default: 0)
 */
+ (void)setStepTarget:(NSInteger)stepTarget calorieTarget:(NSInteger)calorieTarget distanceTarget:
  (NSInteger)distanceTarget sportDurationTarget:(NSInteger)sportDuration sleepDurationTarget:
  (NSInteger)sleepDuration success:(nullable void (^)(void))suc fail:(nullable void (^)(void))fail;
```


3.30 Obtain historical data of timed body temperature measurement

```
/**
 * Obtain historical data of timed body temperature measurement
 */
+ (void)getScheduledTemperatureDataByDayIndex:(NSInteger)dayIndex finished:(nullable void (^)(NSArray * _Nullable temperatureList, NSError * _Nullable error))finished;
```

3.31 Get historical data for manual body temperature measurements

```
/**
 * Get historical data for manual body temperature measurements
 */
+ (void)getManualTemperatureDataByDayIndex:(NSInteger)dayIndex finished:(nullable void (^)(NSArray * _Nullable temperatureList, NSError * _Nullable error))finished;
```

3.32 Get historical data for blood oxygen measurements

```
/**
 * Get historical data for blood oxygen measurements
 */
+ (void)getBloodOxygenDataByDayIndex:(NSInteger)dayIndex finished:(void (^)(NSArray * _Nullable, NSError * _Nullable))finished;
```

3.33 Send firmware file

```
/**
 Send the firmware file and request to use the bin file to upgrade, the result will be processed in the callback

 @param data      OTA binary character stream
 @param start     start sending callback
 @param percentage progress callback
 @param success   success callback
 @param failed    failure callback
 */

+ (void)syncOtaBinData:(NSData *)data
    start:(nullable void (^)(void))start
    percentage:(nullable void (^)(int percentage))percentage
    success:(nullable void (^)(int seconds))success
    failed:(nullable void (^)(NSError *error))failed;
```

3.33.1 Device resource file update (generic)

Demo: QCBandSDKDemo → Home → **Firmware** → **Resource Update**

Sample class: QCSampleResourceUpdateViewController

Resource file update is **not** main firmware OTA (syncOtaBinData). It uses getNeededFileListFinished / syncResourceFileName to fill missing device resource files (e.g. touch assets). Upgrade only when the device reports missing file names.

Integration flow

1. After BLE connected, call setTime and check related features as needed (e.g. QCBandFeatureTouchControl for touch rings).
2. Call getNeededFileListFinished. If the returned list is **empty**, resources are complete — **do not upload**.
3. For each file name in the list: download the file from your server (file name must match **exactly**), then call
syncResourceFileName:name binData:....

Get missing resource file list

```
/**
 * Get the list of resource files missing on the device.
 *
 * @param finished fileList: missing file names; nil/empty = no update needed
 */
+ (void)getNeededFileListFinished:(nullable void (^)(NSArray<NSString *> * _Nullable fileList, NSError * _Nullable error))finished;
```

Upload resource file

```

/**
 * Upload a resource file. `name` must match an entry from getNeededFileListFinished.
 *
 * @param name      File name reported by the device, e.g. RT08_TP_FILE_0x1FD4_250106_01
 * @param data      File binary data (from server download in production)
 * @param start     Start callback
 * @param percentage Progress 0-100
 * @param success   Success callback (seconds)
 * @param failed    Failure callback (e.g. OdmDFUErrorDomain 1003 = device rejected)
 */
+ (void)syncResourceFileName:(NSString *)name
    binData:(NSData *)data
      start:(nullable void (^)(void))start
    percentage:(nullable void (^)(int percentage))percentage
    success:(nullable void (^)(int seconds))success
    failed:(nullable void (^)(NSError *_Nullable error))failed;

```

Example (production pattern)

```

#import <QCBandSDK/QCSDKCmdCreator.h>
#import <QCBandSDK/QCDFU_Utills.h>

- (void)checkAndUpdateResourceIfNeeded {
    [QCSDKCmdCreator setTime:[NSDate date] success:^(NSDictionary *featureList) {
        if (![featureList objectForKey:QCBandFeatureTouchControl] boolValue) {
            return; // Some products need no resource update without touch support
        }
        [QCSDKCmdCreator getNeededFileListFinished:^(NSArray<NSString *> *fileList, NSError *error) {
            if (error || fileList.count == 0) { return; }

            NSString *fileName = fileList.firstObject;
            // TODO: download `fileName` from your server → NSData *data
            NSData *data = [self downloadResourceFileNamed:fileName];

            [QCSDKCmdCreator syncResourceFileName:fileName
                                binData:data
                                start:^( /* show progress UI */ )
                                percentage:^(int p) { /* update progress */ }
                                success:^(int seconds) { /* done */ }
                                failed:^(NSError *err) { /* handle error */ }];

        }];
    } failed:^( /* setTime failed */ );
}

```

Notes

Topic	Description
vs OTA	syncOtaBinData = main .bin firmware; syncResourceFileName = resource files
When to sync	Only when getNeededFileListFinished returns non-empty list
File name	Must match device-reported name exactly
Error 1003	OdmDFUErrorDomain InvalidParameter — file not in missing list, wrong ID/version, or resources already complete
Demo file	qcowireless/TP_resource/RT08_TP_FILE_0x1FD4_250106_01 (bundled sample only)

3.34 Receive a Ring message

```
@interface QCSDKManager : NSObject

/*
 * Receive notifications from Ring, find phone
 */
@property(nonatomic, copy)void(^findPhone)(void);

/*
 * Receive notifications from Ring, enter camera
 */
@property(nonatomic, copy)void(^switchToPicture)(void);

/*
 * Receive notification of Ring, take photo
 */
@property(nonatomic, copy)void(^takePicture)(void);

/*
 * Receive a notification from the Ring to end taking pictures
 */
@property(nonatomic, copy)void(^stopTakePicture)(void);

// singleton class instance
+ (instancetype)shareInstance;

@end
```

3.35 Set/get timed blood oxygen switch status

```
/**
 * Information on setting the timed oximetry function
 * @param feature0n YES: ON; NO: OFF
 */

+ (void)setSchedualB0Info0n:(BOOL)feature0n success:(nullable void (^)(BOOL feature0n))success fail:(void (^)(void))fail;

/**
 * Get information about the timed oximetry function
 * @param success feature0n YES: 开启; NO: 关闭
 */

+ (void)getSchedualB0InfoSuccess:(nullable void (^)(BOOL feature0n))success fail:(void (^)(void))fail;
```

3.36 Send measurement commands (commands are encapsulated in QCSDKManager)

```
typedef NS_ENUM(NSInteger, QCMeasuringType) {
    QCMeasuringTypeUnkown = -1,
    QCMeasuringTypeHeartRate = 0,    //Heart rate measurement
    QCMeasuringTypeBloodPressue,    //blood pressure measurement
    QCMeasuringTypeBloodOxygen,    //blood oxygen measurement
    QCMeasuringTypeOneKeyMeasure,    //One-click measurement
    QCMeasuringTypeStress,
    QCMeasuringTypeBloodGlucose,
    QCMeasuringTypeHRV,
    QCMeasuringTypeBodyTemperature,    //blood temperature measurement
    QCMeasuringTypeThreeValueBodyTemperature,    //blood temperature measurement
    QCMeasuringTypeOneKeyMeasureHeartRate,    //Heart rate onekey measurement
    QCMeasuringTypeHeartRateRaw,    //Heart rate measurement raw data(PPG)
    QCMeasuringTypeBloodOxygenRaw,    //oxygen measurement raw data(PPG)
    QCMeasuringTypeCount,
};

//The measurement result is the result in the handle callback
//When measuring heart rate, result returns NSNumber: @(60)
//When measuring blood pressure, the result returned is NSDictionary:@{@"sbp":@"120",@"dbp":@"60"}
//When measuring blood oxygen, the result returns NSNumber: @(98)

/**
 * @brief Start a health measurement with the built-in default timeout.
 * 使用内置默认超时时间开始健康测量。
 *
 * ##### Default timeout by type (unit: seconds):
 * ##### 各类型默认超时时间 (单位: 秒):
 * - Most types: 30
 * - 大多数类型: 30
 * - QCMeasuringTypeHRV: 80
 * - QCMeasuringTypeHRV: 80
 *
 * @param type Measurement type. See QCMeasuringType.
 * 测量类型, 参见 QCMeasuringType.
 * @param measuring Real-time value callback while measuring. Result type depends on type.
 * 测量过程中的实时数值回调。返回结果类型取决于测量类型。
 * @param handle Final result callback.
 * 最终结果回调。
 *
 * error.code: -1 start command failed, -2 end command failed,
 * -3 device not worn properly, -4 uncalibrated.
 * error.code: -1 发送开始指令失败, -2 发送结束指令失败,
 * -3 设备未正确佩戴, -4 未校准。
 */
- (void)startToMeasuringWithOperateType:(QCMeasuringType)type measuringHandle:(void(^)(id _Nullable
result))measuring completedHandle:(void(^)(BOOL isSuccess,id _Nullable result, NSError * _Nullable
error))handle;

/**
 * @brief Start a health measurement with a custom timeout.
 * 使用自定义超时时间开始健康测量。
 *
 * ##### Important: timeout unit is seconds. Passing a wrong unit/value may stop the
 * measurement too early (no valid result) or keep the sensor running longer than expected.
 * ##### 重要: timeout 单位为秒。传错单位或数值可能导致测量过早结束 (无有效结果),
 * 或传感器运行时间超出预期。
 *
 * ##### Recommended values:
 * ##### 推荐取值:
 * - General measurements (HR / BP / SpO2 / stress / glucose / temperature): 30
 * - 常规测量 (心率 / 血压 / 血氧 / 压力 / 血糖 / 体温): 30
 * - QCMeasuringTypeHRV: 80
 * - QCMeasuringTypeHRV: 80
 * - QCMeasuringTypeHeartRateRaw / QCMeasuringTypeBloodOxygenRaw (PPG): 90 or larger;
 * for continuous long capture, pass a large value such as 36000
 * - QCMeasuringTypeHeartRateRaw / QCMeasuringTypeBloodOxygenRaw (PPG): 90 或更大;
 * 长时间连续采集可传较大值, 例如 36000
 *
 * @param type Measurement type. See QCMeasuringType.
 * 测量类型, 参见 QCMeasuringType.
 * @param timeout Measurement duration in seconds.
 * Pass >= 0 to use this custom duration.
 * Pass < 0 to fall back to the built-in default for the type
 * (30 for most types, 80 for HRV, 90 for heart-rate raw PPG).
 * 测量时长, 单位: 秒。
 * 传 >= 0 使用该自定义时长。
 * 传 < 0 则回退到该类型的内置默认值
 */
```

```

* (大多数类型 30, HRV 80, 心率 PPG 原始数据 90)。
* @param measuring Real-time value callback while measuring. Result type depends on type.
* 测量过程中的实时数值回调。返回结果类型取决于测量类型。
* @param handle Final result callback.
* 最终结果回调。
*
* error.code: -1 start command failed, -2 end command failed,
* -3 device not worn properly, -4 uncalibrated.
* error.code: -1 发送开始指令失败, -2 发送结束指令失败,
* -3 设备未正确佩戴, -4 未校准。
*/
- (void)startToMeasuringWithOperateType:(QCMeasuringType)type timeout:(NSInteger)timeout measuringHandle:(void(^)(
    id _Nullable result))measuring completedHandle:(void(^)(BOOL isSuccess,id _Nullable result, NSError *
    _Nullable error))handle;

/**
* @brief Stop an in-progress health measurement.
* 停止正在进行的健康测量。
*
* @param type Measurement type that was started. See QCMeasuringType.
* 已开始的测量类型, 参见 QCMeasuringType.
* @param handle Completion callback.
* 完成回调。
*
* error.code: -1 failed to send the end command.
* error.code: -1 发送结束指令失败。
*/
- (void)stopToMeasuringWithOperateType:(QCMeasuringType)type completedHandle:(void(^)(BOOL isSuccess, NSError
    *error))handle;

```

3.37 Sleep protocol (get a day to today)

```

// Main sleep from fromDayIndex through today.
+ (void)getSleepDetailDataFromDay:(NSInteger)fromDayIndex
    sleepDats:(nullable void (^)(NSDictionary<NSString *,
        NSArray<QCSleepModel *> *> *sleeps))items
    fail:(nullable void (^)(void))fail;

// Main sleep and naps from fromDayIndex through today.
+ (void)getFullDaySleepDetailDataFromDay:(NSInteger)fromDayIndex
    sleepDats:(nullable void (^)(NSDictionary<NSString *,
        NSArray<QCSleepModel *> *> *sleeps,
        NSDictionary<NSString *,
        NSArray<QCSleepModel *> *> *naps))items
    fail:(nullable void (^)(void))fail;

```

Dictionary keys are day-index strings such as @"0" and @"1", not date strings. The device may omit days with no sleep data, so keys are not guaranteed to be consecutive.

3.38 RealTime HeartRate Measuring

```

typedef enum {
    QCBandRealTimeHeartRateCmdTypeStart, // Start real-time heart rate measurement
    QCBandRealTimeHeartRateCmdTypeEnd,   // End real-time heart rate measurement
    QCBandRealTimeHeartRateCmdTypeHold,   // Keep-alive during continuous measurement
} QCBandRealTimeHeartRateCmdType;

/**
* RealTime HeartRate Measuring
* 实时心率测量
*
* @param type :common type
* @param finished :finish callback
*/
+ (void)realTimeHeartRateWithCmd:(QCBandRealTimeHeartRateCmdType)type finished:(nullable void (^)(BOOL))finished;

```

3.39 Set Sport Mode State (Only Ring support)

```

/// Set Sport Mode State
///
/// - Parameters:
///   - sportType: type
///   - state: state
///   - finished: finished callback
+ (void)operateSportModeWithType:(OdmSportPlusExerciseModelType)sportType state:(QCSportState)state finish:(void(^)(i

```

Get callback:

```
[QCSDKManager sharedInstance].currentSportInfo = ^(QCSportInfoModel * _Nonnull sportInfo) {
    NSLog(@"sportType:%zd,duration:%zd,state:%u,hr:%zd,step:%zd,calorie(unit:calorie):%zd,distance(unit:meter):%zd",sportInfo.sportType,sportInfo.duration,sportInfo.state,sportInfo.hr,sportInfo.steps,sportInfo.calorie,sportInfo.distance);
};
```

3.40 Get Schedual Stress Datas (Only Ring support)

```
/// Get Schedual Stress Datas (Only Ring Support)
///
/// - Parameters:
/// - dates: 0-6,0:today,1:yesterday...
/// - finished: finished callback
+ (void)getSchedualStressDataWithDates:(NSArray<NSNumber*> *)dates finished:(void (^)(NSArray * _Nullable, NSError * _Nullable))finished;

/// Get Schedual Stress Status
///
/// - Parameter finished: finished callback
+ (void)getSchedualStressStatusWithFinshed:(nullable void (^)(BOOL,NSError * _Nullable error))finished;

/// Set Schedual Stress Status
///
/// - Parameters:
/// - enable:YES:0n,N0:0ff
/// - finished: finished callback
+ (void)setSchedualStressStatus:(BOOL)enable finshed:(nullable void (^)(NSError * _Nullable error))finished;
```

3.42 Get Schedual HRV Data (Only Ring Support)

Demo: QCSampleHRVViewController — getHRVSamplesWithDayIndexes:finished:

Import result models:

```
#import <QCBandSDK/QCHRVDayModel.h>
#import <QCBandSDK/QCHRVSampleModel.h>
```

Get HRV samples with absolute timestamps (preferred)

```
/**
 * Get HRV samples with absolute timestamps (preferred).
 * SDK normalizes device raw slots to a 5-minute / 288-slot full-day timeline.
 *
 * sample.time = dayStart + index * 5min
 * sample.value == 0 means no valid reading at that slot.
 *
 * @param dayIndexes 0-6, 0=today, 1=yesterday, ...
 * @param finished NSArray<QCHRVDayModel*> and error
 */
+ (void)getHRVSamplesWithDayIndexes:(NSArray<NSNumber*> *)dayIndexes
    finished:(void (^)(NSArray<QCHRVDayModel*> * _Nullable days, NSError * _Nullable error))finished;
```

Get scheduled HRV data (legacy raw array)

```
/// Legacy. Prefer +getHRVSamplesWithDayIndexes:finished:.
+ (void)getSchedualHRVDataWithDates:(NSArray<NSNumber*> *)dates finished:(void (^)(NSArray * _Nullable, NSError * _Nullable))finished
    __deprecated_msg("Use +getHRVSamplesWithDayIndexes:finished;; each QCHRVSampleModel provides time (288 slots/day at 5-min grid).");
```

Get / set scheduled HRV switch

```
+ (void)getSchedualHRVWithFinshed:(nullable void (^)(BOOL,NSError * _Nullable error))finished;

+ (void)setSchedualHRVStatus:(BOOL)enable finshed:(nullable void (^)(NSError * _Nullable error))finished;
```

Result models

QCHRVDayModel

Property	Type	Description
date	NSString *	Date string, format: yyyy-MM-dd
timestamp	uint32_t	Absolute Unix of local day start (00:00:00)
intervalMinutes	NSInteger	Timeline slot interval; always 5 (288 slots/day)
deviceIntervalMinutes	NSInteger	Device storage interval (e.g. 30 for DS500)
samples	NSArray<QCHRVSampleModel >	Full-day timeline; value == 0 = no reading

QCHRVSampleModel

Property	Type	Description
time	NSDate *	Absolute sample time (local calendar)
unixTimestamp	uint32_t	Unix seconds for time
value	NSInteger	HRV in milliseconds; 0 = no valid reading

Example

```
[QCSDKCmdCreator getScheduledHRVWithFinished:^(BOOL isOn, NSError *error) {
    if (error || !isOn) { return; }
    [QCSDKCmdCreator getHRVSamplesWithDayIndexes:@[(0), @(1)] finished:^(NSArray<QCHRVDayModel * > *days, NSError *err) {
        if (err) { return; }
        for (QCHRVDayModel *day in days) {
            for (QCHRVSampleModel *sample in day.samples) {
                if (sample.value > 0) {
                    // sample.time, sample.value (ms)
                }
            }
        }
    }];
}];
```

Prefer getHRVSamplesWithDayIndexes:finished: for App UI; sample times are absolute local-day slots on a 5-minute grid.

3.43 Get Touch Control Type

```
/// Get Touch Control Type
///
/// @param finished : callback-> type:QCTouchGestureControlType ,strength:1-10
+ (void)getTouchControlFinished:(nullable void (^)(QCTouchGestureControlType,NSInteger,NSError *_Nullable error))finished;

/// Set Touch Control Type
/// @param type : type
/// @param strength :1-10
/// @param finished :callback
+ (void)setTouchControl:(QCTouchGestureControlType)type strength:(NSInteger)strength finished:(nullable void (^)(NSError *_Nullable error))finished;

/// Get Gesture Control Type
///
/// @param finished : callback-> type:QCTouchGestureControlType ,strength:1-10
+ (void)getGestureControlFinished:(nullable void (^)(QCTouchGestureControlType,NSInteger,NSError *_Nullable error))finished;

/// Set Gesture Control Type
/// @param type : type
/// @param strength :1-10
/// @param finished :callback
+ (void)setGestureControl:(QCTouchGestureControlType)type strength:(NSInteger)strength finished:(nullable void (^)(NSError *_Nullable error))finished;
```

3.43 Wearing Calibration

```
/// Wearing Calibration
///
/// @param type 1->Start calibration (reset ring data), 2->End calibration, 3->Get single data, 4->Power
///           consumption mode, 5->Stop power consumption, 6->App starts calibration
/// @param finished finished callback
+ (void)wearCalibration:(NSInteger)type finished:(nullable void (^)(NSError *_Nullable error))finished;
```

3.44 Get Sedentary Reminder

```
/// Get Sedentary Reminder (Only Ring Support)
///
/// @param fromDayIndex :0->Today,1->Yesterday,2->The day before yesterday ...
/// @param finished : callback
+ (void)getSedentaryReminderFromDay:(NSInteger)fromDayIndex finished:(nullable void (^)(NSDictionary
<NSString*,NSArray<QCSedentaryModel*>*_Nullable datas, NSError *_Nullable error))finished;
```

3.45 Set and get device uuid

```
/// Sets the device UUID.
///
/// @param uuid Device identifier string, must be less than 10 characters.
/// Note: This feature is only supported by certain devices.
/// @param suc Callback invoked when the operation succeeds.
/// @param fail Callback invoked when the operation fails.
+ (void)setUUID:(NSString *)uuid success:(void (^)(void))suc failed:(void (^)(void))fail;
```

```
/**
End Broadcast

@param suc Callback invoked when the operation succeeds.
@param fail Callback invoked when the operation fails.
*/
+ (void)endBroadcast:(void (^)(void))suc
failed:(void (^)(void))fail;
```


3.45 Schedual Blood Oxygen With Time Interval

```
/**
 * @brief Configure scheduled blood oxygen (B0) monitoring
 *
 * @param feature0n YES to enable scheduled B0 monitoring, NO to disable
 * @param timeInterval Monitoring interval in minutes
 * @param success Success callback
 * @param fail Failure callback
 */
+ (void)setSchedualB0Info0n:(BOOL)feature0n
    timeInterval:(NSInteger)timeInterval
    success:(void (^)(void))success
    fail:(void (^)(void))fail;

/**
 * @brief Retrieve scheduled blood oxygen (B0) monitoring configuration
 *
 * @param success Success callback with two parameters:
 *             BOOL → Indicates whether scheduled B0 monitoring is enabled
 *             NSInteger → Monitoring interval in minutes
 * @param fail Failure callback
 */
+ (void)getSchedualB0InfoWithIntervalSuccess:(void (^)(BOOL, NSInteger))success
    fail:(void (^)(void))fail;

/**
 * @brief Get blood oxygen (B0) data for a specific day
 *
 * @param dayIndex Index of the day (0 = today, 1 = yesterday, and so on)
 * @param finished Completion callback with three parameters:
 *             NSInteger → Number of data entries
 *             NSArray → Array of B0 data (nullable)
 *             NSError → Error information (nullable, non-nil if failed)
 */
+ (void)getBloodOxygenDataWithIntervalByDayIndex:(NSInteger)dayIndex
    finished:(void (^)(NSInteger, NSArray * _Nullable, NSError *
        _Nullable))finished;
```

3.46 End Broadcast

```
/**
 End Broadcast

 @param suc Callback invoked when the operation succeeds.
 @param fail Callback invoked when the operation fails.
 */
+ (void)endBroadcast:(void (^)(void))suc
    failed:(void (^)(void))fail;
```

3.47 set Schedual info

```
// / Set setSchedual Info (Only Ring Support)
// /
// / @param type :data type
// / @param feature0n :
// /
// / @param success : callback
+ (void)setSchedualInfoType:(SchedualInfoType)type feature0n:(BOOL)feature0n calibrate:(NSInteger)calibrate
    success:(nullable void (^)(void))success fail:(nullable void (^)(void))fail;

/// Set setSchedual Info with interval (Only Ring Support)
///
/// @param type :data type
/// @param interval : NSInteger type
/// @param feature0n :
///
/// @param success : callback
+ (void)setSchedualInfoType:(SchedualInfoType)type feature0n:(BOOL)feature0n calibrate:(NSInteger)calibrate
    interval:(NSInteger)interval success:(nullable void (^)(void))success fail:(nullable void (^)(void))fail;
```


3.52 setSchedualHeartRateStatus(all parameters)

```
/**
 * @brief Set scheduled heart rate monitoring configuration with all parameters
 *
 * @param enable Whether to enable scheduled heart rate monitoring. YES = enabled, NO = disabled
 * @param interval Heart rate measurement interval in minutes
 * @param maxHrTip Automatic heart rate alarm maximum threshold (bpm)
 * @param minHrTip Automatic heart rate alarm minimum threshold (bpm)
 * @param success Success callback with two parameters:
 *     BOOL → Indicates whether the feature is enabled after setting
 *     NSInteger → Heart rate measurement interval after setting
 * @param fail Failure callback
 */
+ (void)setSchedualHeartRateStatus:(BOOL)enable interval:(NSInteger)interval maxHrTip:(NSInteger)maxHrTip
    minHrTip:(NSInteger)minHrTip success:(nullable void (^)(BOOL enable, NSInteger interval))success fail:
    (nullable void (^)(void))fail;
```

3.53 setSchedualHeartRateStatus(enable and interval)

```
/**
 * @brief Set scheduled heart rate monitoring configuration with interval only
 *
 * @param enable Whether to enable scheduled heart rate monitoring. YES = enabled, NO = disabled
 * @param interval Heart rate measurement interval in minutes
 * @param success Success callback with two parameters:
 *     BOOL → Indicates whether the feature is enabled after setting
 *     NSInteger → Heart rate measurement interval after setting
 * @param fail Failure callback
 */
+ (void)setSchedualHeartRateStatus:(BOOL)enable interval:(NSInteger)interval success:(nullable void (^)(BOOL
    enable, NSInteger interval))success fail:(nullable void (^)(void))fail;
```

3.54 getSchedualHeartRateInfoWithSuccess

```
/**
 * @brief Get the Info of the scheduled heart rate function
 *
 * @param success Success callback with five parameters:
 *     BOOL enable → Indicates whether scheduled heart rate monitoring is enabled
 *     NSInteger interval → Heart rate measurement interval in minutes
 *     NSInteger minInterval → Minimum heart rate measurement duration
 *     NSInteger minHrTip → Automatic heart rate alarm minimum threshold (bpm)
 *     NSInteger maxHrTip → Automatic heart rate alarm maximum threshold (bpm)
 * @param fail Failure callback
 */
+ (void)getSchedualHeartRateInfoWithSuccess:(nullable void (^)(BOOL enable, NSInteger interval, NSInteger
    minInterval, NSInteger minHrTip, NSInteger maxHrTip))success fail:(nullable void (^)(void))fail;
```

3.55 Get Pressure / Emotion History Data

This API is used to obtain **pressure** or **emotion** historical data (long-packet history APIs).

Note: This is different from the short-packet stress API `getSchedualStressDataWithDates:`. These APIs return raw byte-array pressure data or emotion value/status pairs.

Import the result models:

```
#import <QCBandSDK/QCPressureDayModel.h>
#import <QCBandSDK/QCPressureSampleModel.h>
#import <QCBandSDK/QCOtherDataPressureModel.h> // legacy
#import <QCBandSDK/QCOtherDataEmotionModel.h>
#import <QCBandSDK/QCEmotionItemModel.h>
```

Get pressure samples with absolute timestamps (preferred)

```
/// Preferred. Each QCPressureSampleModel.time is absolute (like QCTemperatureModel.time).
/// sample.value == 0 means no valid reading at that slot.
+ (void)getPressureSamplesWithDayIndexes:(NSArray<NSNumber *>)dayIndexes
    finished:(void (^)(NSArray<QCPressureDayModel *> * _Nullable days, NSError *
    _Nullable error))finished;
```

Get pressure data raw array (legacy)

```
/// Legacy raw values[]. Prefer getPressureSamplesWithDayIndexes:finished:.
+ (void)getOtherDataPressureWithDayIndexes:(NSArray<NSNumber *> *)dayIndexes finished:(void (^)(
    NSArray<QCOtherDataPressureModel *> * _Nullable, NSError * _Nullable))finished
    __deprecated_msg("Use +getPressureSamplesWithDayIndexes:finished:; each QCPressureSampleModel provides time
        (like QCTemperatureModel).");
```

Get emotion data

```
/// Get Other Data Emotion Datas
///
/// - Parameters:
///   - dayIndexes: 0-6, 0:today, 1:yesterday...
///   - finished: finished callback
+ (void)getOtherDataEmotionWithDayIndexes:(NSArray<NSNumber *> *)dayIndexes finished:(void (^)(
    NSArray<QCOtherDataEmotionModel *> * _Nullable, NSError * _Nullable))finished;
```

Timestamp convention

The 4-byte day timestamp on the wire is a **wall-clock Unix** value (same rule as scheduled heart-rate history), **not** the absolute UTC instant of local midnight:

```
wireUnix = localDayStart.timeIntervalSince1970
    + [NSTimeZone systemTimeZone].secondsFromGMT
```

Example (UTC+8, local day 2026-07-25 00:00):

Role	Value	Meaning
Wire / protocol	1784937600	Wall-clock day start (UTC decode shows 07-25 00:00)
Absolute local midnight	1784908800	True UTC of East-8 midnight (07-24 16:00 UTC)

Pressure and emotion history use the same timestamp wire rule as scheduled HR. QCPressureDayModel.timestamp / QCPressureSampleModel.time expose the **absolute** local day start for App display (00:00-23:55 in the phone time zone).

Result models

QCPressureDayModel (preferred)

Property	Type	Description
date	NSString *	Date string, format: yyyy-MM-dd
timestamp	uint32_t	Absolute Unix of local day start (00:00:00)
intervalMinutes	NSInteger	Sampling interval in minutes
samples	NSArray<QCPressureSampleModel >	Full-day timeline; value == 0 = no reading. If device returns dataLength=0, SDK still expands a zero-filled day

QCOtherDataPressureModel (legacy)

Property	Type	Description
timestamp	uint32_t	Wire / wall-clock Unix of the requested day (same as protocol)
date	NSString *	Date string, format: yyyy-MM-dd
interval	NSInteger	Sampling interval
values	NSArray<NSNumber >	Pressure values (0~255 per sample); empty when dataLength=0

QCOtherDataEmotionModel

Property	Type	Description
timestamp	uint32_t	Wire / wall-clock Unix of the requested day (same as protocol)
date	NSString *	Date string, format: yyyy-MM-dd
interval	NSInteger	Sampling interval
emotions	NSArray<QCEmotionItemModel >	Emotion items

QCEmotionItemModel

Property	Type	Description
value	NSInteger	Emotion value
status	QCEmotionStatus	Emotion status enum

QCEmotionStatus values: QCEmotionStatusNull(0), QCEmotionStatusExcited(1), QCEmotionStatusHappy(2), QCEmotionStatusPleasant(3), QCEmotionStatusCalm(4), QCEmotionStatusNervous(5), QCEmotionStatusConfused(6), QCEmotionStatusChallenged(7), QCEmotionStatusUncomfortable(8), QCEmotionStatusSad(9), QCEmotionStatusAnxious(10).

Example

```
// Get today's pressure data
[QCSDKCmdCreator getPressureSamplesWithDayIndexes:@[@(0)] finished:^(NSArray<QCPressureDayModel *> *days, NSError
 *error) {
    if (error) {
        return;
    }
    for (QCPressureDayModel *day in days) {
        for (QCPressureSampleModel *sample in day.samples) {
            if (sample.value == 0) { continue; }
            NSLog(@"%@ value=%ld", sample.time, (long)sample.value);
        }
    }
}];

// Get yesterday's emotion data (enable emotion switch first, see 3.56)
[QCSDKCmdCreator setSchedualEmotionStatus:YES finished:^(NSError *error) {
    if (error) {
        return;
    }
    [QCSDKCmdCreator getOtherDataEmotionWithDayIndexes:@[@(1)] finished:^(NSArray<QCOtherDataEmotionModel *>
 *models, NSError *error) {
        if (error) {
            return;
        }
        for (QCOtherDataEmotionModel *model in models) {
            NSLog(@"date:%@, interval:%ld, emotions count:%lu", model.date, (long)model.interval, (unsigned
 long)model.emotions.count);
        }
    }
}];
```

Note:

- dayIndexes supports 0~6 (0 = today, 1 = yesterday, and so on). Cross-year dates are handled automatically.
- model.date is calculated from the requested dayIndex on the App side.
- Wire timestamp uses wall-clock Unix (date + secondsFromGMT), consistent with scheduled heart-rate history.
- Prefer getPressureSamplesWithDayIndexes:finished: for App UI; sample times are absolute local-day slots.
- For emotion data, turn on the emotion auto-measurement switch (see 3.56) before reading; otherwise the payload may be empty.

3.56 Set / Get Schedual Emotion Status

This API controls the **emotion auto-measurement switch** (SchedualInfoTypeEmotion).

It uses the same API family as setSchedualInfoType: (e.g. body temperature, blood glucose). Stress switch (setSchedualStressStatus:) is a different feature.

```

/// Get Scheduling Emotion Status
///
/// - Parameter finished: finished callback, isOn = auto measurement enabled
+ (void)getSchedulingEmotionStatusWithFinished:(nullable void (^)(BOOL isOn, NSError *_Nullable error))finished;

/// Set Scheduling Emotion Status
///
/// - Parameters:
///   - enable: YES:On, NO:Off
///   - finished: finished callback
+ (void)setSchedulingEmotionStatus:(BOOL)enable finished:(nullable void (^)(NSError *_Nullable error))finished;

```

Equivalent generic API:

```

+ (void)setSchedulingInfoType:(SchedulingInfoType)type featureOn:(BOOL)featureOn calibrate:(NSInteger)calibrate
  success:(nullable void (^)(void))success fail:(nullable void (^)(void))fail;

+ (void)getSchedulingInfoType:(SchedulingInfoType)type success:(void (^)(BOOL, NSInteger))success fail:(void (^)(
  (void)))fail;

```

Use SchedulingInfoTypeEmotion for the type parameter.

Example

```

// Turn on emotion auto-measurement
[QCSDKCmdCreator setSchedulingEmotionStatus:YES finished:^(NSError *error) {
    if (error) {
        return;
    }
    [QCSDKCmdCreator getSchedulingEmotionStatusWithFinished:^(BOOL isOn, NSError *error) {
        NSLog(@"emotion switch isOn:%d", isOn);
    }];
}];

// Then read emotion history data (see 3.55)
[QCSDKCmdCreator getOtherDataEmotionWithDayIndexes:@[0]] finished:^(NSArray<QCOtherDataEmotionModel *> *models,
  NSError *error) {
    // handle models
}];

```

3.57 Send Notification Function Event

This API sends **notification function events** to the device, including preset event notifications and custom light/vibration notifications.

Event types (QCNotificationEventType)

Value	Enum	Description
1	QCNotificationEventTypeDoubleClick	Double-click button event
2	QCNotificationEventTypePairing	Pairing event
3	QCNotificationEventTypeMissYou	"I miss you" event
4	QCNotificationEventTypeLoveYou	"I love you" event
5	QCNotificationEventTypeNeedYou	"Need you" event
6	QCNotificationEventTypeSad	"Sad" event
8	QCNotificationEventTypeCustomLight	Custom light notification
9	QCNotificationEventTypeCustomVibration	Custom vibration notification

For preset events, only the event type is required; other parameters are handled by the SDK.

Custom light action (QCNotificationLightAction)

Value	Enum	Description
0	QCNotificationLightActionNull	NULL
1	QCNotificationLightActionStop	Stop
2	QCNotificationLightActionGreen	Green light
3	QCNotificationLightActionRed	Red light

Custom vibration action (QCNotificationVibrationAction)

Value	Enum	Description
0	QCNotificationVibrationActionNull	NULL
1	QCNotificationVibrationActionStop	Stop
2	QCNotificationVibrationActionStart	Start

Send preset notification event

```
/**
 * Send notification function event
 * 发送通知功能事件
 */
+ (void)sendNotificationEvent:(QCNotificationEventType)eventType success:(nullable void (^)(void))suc fail:
    (nullable void (^)(void))fail;
```

Send custom light notification

```
/**
 * Send custom light notification
 * 发送自定义亮灯通知
 * @param onDuration    Light-on duration, unit: 100ms, range: 1~127
 * @param pauseDuration Pause duration, unit: 100ms
 */
+ (void)sendCustomLightNotificationWithPriority:(NSInteger)priority
    action:(QCNotificationLightAction)action
    cycleCount:(NSInteger)cycleCount
    onDuration:(NSInteger)onDuration
    pauseDuration:(NSInteger)pauseDuration
    success:(nullable void (^)(void))suc
    fail:(nullable void (^)(void))fail;
```

Send custom vibration notification

```
/**
 * Send custom vibration notification
 * 发送自定义震动通知
 * @param vibrateDuration Vibration duration, unit: 100ms, range: 1~127
 * @param pauseDuration  Pause duration, unit: 100ms
 */
+ (void)sendCustomVibrationNotificationWithPriority:(NSInteger)priority
    action:(QCNotificationVibrationAction)action
    cycleCount:(NSInteger)cycleCount
    vibrateDuration:(NSInteger)vibrateDuration
    pauseDuration:(NSInteger)pauseDuration
    success:(nullable void (^)(void))suc
    fail:(nullable void (^)(void))fail;
```

Example

```
// Preset event
[QCSDKCmdCreator sendNotificationEvent:QCNotificationEventTypeSad success:^(
    NSLog(@"Send notification event successfully");
} fail:^(
    NSLog(@"Failed to send notification event");
)];

// Custom red light: priority=1, 3 cycles, 1.0s on, 0.5s pause
[QCSDKCmdCreator sendCustomLightNotificationWithPriority:1
                 action:QCNotificationLightActionRed
                 cycleCount:3
                 onDuration:10
                 pauseDuration:5
                 success:^(
    NSLog(@"Send custom light notification successfully");
} fail:^(
    NSLog(@"Failed to send custom light notification");
)];

// Custom vibration: priority=1, 2 cycles, 1.2s vibrate, 1.0s pause
[QCSDKCmdCreator sendCustomVibrationNotificationWithPriority:1
                 action:QCNotificationVibrationActionStart
                 cycleCount:2
                 vibrateDuration:12
                 pauseDuration:10
                 success:^(
    NSLog(@"Send custom vibration notification successfully");
} fail:^(
    NSLog(@"Failed to send custom vibration notification");
)];
```

3.58 Get/Set Music Control Switch

Demo: QCSampleNotificationsViewController

```
/**
 * Get music-control switch status.
 * @param enable Known state hint
 */
+ (void)getMusicControlStatusWithCurrentState:(BOOL)enable
    success:(nullable void (^)(BOOL enable))success
    fail:(nullable void (^)(void))fail;

/**
 * Set music-control switch status.
 * @param enable YES to enable music control, NO to disable
 */
+ (void)setMusicControlStatus:(BOOL)enable
    success:(nullable void (^)(BOOL enable))success
    fail:(nullable void (^)(void))fail;
```

Example

```
[QCSDKCmdCreator getMusicControlStatusWithCurrentState:YES success:^(BOOL enable) {
    NSLog(@"music control=%d", enable);
} fail:^({});

[QCSDKCmdCreator setMusicControlStatus:YES success:^(BOOL enable) {
    NSLog(@"set music control=%d", enable);
} fail:^({});
```

3.59 Get/Set AGPS Sync Switch

Demo: QCSampleAdvancedViewController

Some devices (e.g. certain rings) may not support this command; timeout failure is expected.


```

/**
 * Get AGPS switch status.
 * @param enable Known state hint
 */
+ (void)getAGPSStatusWithCurrentState:(BOOL)enable
    success:(nullable void (^)(BOOL enable))success
    fail:(nullable void (^)(void))fail;

/**
 * Set AGPS switch status.
 * @param enable YES to enable AGPS, NO to disable
 */
+ (void)setAGPSStatus:(BOOL)enable
    success:(nullable void (^)(BOOL enable))success
    fail:(nullable void (^)(void))fail;

```

Example

```

[QCSDKCmdCreator getAGPSStatusWithCurrentState:YES success:^(BOOL enable) {
    NSLog(@"AGPS=%d", enable);
} fail:^({});

[QCSDKCmdCreator setAGPSStatus:YES success:^(BOOL enable) {
    NSLog(@"set AGPS=%d", enable);
} fail:^({});

```

3.60 Get/Set Intelligent Alarm (PXP) Delay

Demo: QCSampleAdvancedViewController
Some devices may not support this command.

```

/**
 * Get intelligent (PXP) alarm delay info.
 * Success callback: (featureOn, delaySeconds)
 */
+ (void)getPxpAlarmDelay:(nullable void (^)(BOOL featureOn, NSInteger delaySeconds))success
    fail:(nullable void (^)(void))fail;

/**
 * Set intelligent (PXP) alarm delay info.
 * @param on YES to enable
 * @param delaySeconds Delay in seconds (0~255)
 */
+ (void)setPxpAlarmDelay:(BOOL)on
    delaySeconds:(NSInteger)delaySeconds
    success:(nullable void (^)(BOOL featureOn, NSInteger realDelaySeconds))success
    fail:(nullable void (^)(void))fail;

```

Example

```

[QCSDKCmdCreator getPxpAlarmDelay:^(BOOL featureOn, NSInteger delaySeconds) {
    NSLog(@"pxp on=%d delay=%ld", featureOn, (long)delaySeconds);
} fail:^({});

[QCSDKCmdCreator setPxpAlarmDelay:YES delaySeconds:5 success:^(BOOL featureOn, NSInteger realDelay) {
    NSLog(@"set pxp on=%d delay=%ld", featureOn, (long)realDelay);
} fail:^({});

```

3.61 Get Extended Device Feature Config

Demo: QCSampleAdvancedViewController
Complements the base featureList from setTime.

```

/**
 * Get extended device feature configuration dictionary.
 */
+ (void)getDeviceFeatureConfigSuccess:(nullable void (^)(NSDictionary *featureList))success
    fail:(nullable void (^)(void))fail;

```

Example

```
[QCSDKCmdCreator getDeviceFeatureConfigSuccess:^(NSDictionary *featureList) {
    NSLog(@"featureConfig=%@", featureList);
} fail:^({});
```

3.62 Get UI Personalization Resource Types

Demo: QCSampleAdvancedViewController

Query resource types for standby / boot / shutdown UI pages. Some devices may not support this.

```
/**
 * Get resource types for UI pages.
 * @param uiTypes UI type array. See ODM_RES_UIType
 * @param success Resource type array. See ODM_RES_ResourceType (aligned with uiTypes)
 */
+ (void)getResourceWithUITypes:(NSArray<NSNumber *> *)uiTypes
    success:(nullable void (^)(NSArray<NSNumber *> *resourceTypes))success
    fail:(nullable void (^)(void))fail;
```

ODM_RES_UIType	Description
ODM_RES_UIType_StandBy	Standby
ODM_RES_UIType_Boot	Boot
ODM_RES_UIType_ShutDown	Shutdown

ODM_RES_ResourceType	Description
ODM_RES_ResourceType_Default	Default / none
ODM_RES_ResourceType_Image	Image
ODM_RES_ResourceType_Text	Text

Example

```
NSArray *uiTypes = @[
    @(ODM_RES_UIType_StandBy),
    @(ODM_RES_UIType_Boot),
    @(ODM_RES_UIType_ShutDown)
];
[QCSDKCmdCreator getResourceWithUITypes:uiTypes success:^(NSArray<NSNumber *> *resourceTypes) {
    NSLog(@"resourceTypes=%@", resourceTypes);
} fail:^({});
```

3.63 Get/Set Device Brightness

Demo: QCSampleDisplayViewController

```
/**
 * Get device brightness level.
 * @param lightLevel Known brightness hint (1-10 => 10%-100%)
 */
+ (void)getDeviceLightLevelWithCurrentLevel:(NSInteger)lightLevel
    success:(nullable void (^)(NSInteger lightLevel))success
    fail:(nullable void (^)(void))fail;

/**
 * Set device brightness level.
 * @param lightLevel Brightness level, 1-10 => 10%-100%
 */
+ (void)setDeviceLightLevel:(NSInteger)lightLevel
    success:(nullable void (^)(NSInteger lightLevel))success
    fail:(nullable void (^)(void))fail;
```

3.64 Get/Set Screen Orientation / Wear Hand

Demo: QCSampleDisplayViewController

```

/**
 * Get screen orientation / wear-hand info.
 * @param portraitOrientation Known portrait hint (YES=portrait, NO=landscape)
 * @param leftHandWear Known left-hand hint (effective in landscape)
 */
+ (void)getScreenOrientationInfoWithPortrait:(BOOL)portraitOrientation
    leftHandWear:(BOOL)leftHandWear
    success:(nullable void (^)(BOOL portraitOrientation, BOOL
    leftHandWear))success
    fail:(nullable void (^)(void))fail;

/**
 * Set screen orientation / wear-hand info.
 */
+ (void)setScreenOrientationInfoWithPortrait:(BOOL)portraitOrientation
    leftHandWear:(BOOL)leftHandWear
    success:(nullable void (^)(BOOL portraitOrientation, BOOL
    leftHandWear))success
    fail:(nullable void (^)(void))fail;

```

3.65 Get/Set Raise-to-Wake Clock Homepage

Demo: QCSampleDisplayViewController

```

+ (void)getScreenHomePageShowingClockWithCurrentState:(BOOL)showing
    success:(nullable void (^)(BOOL showingClockOnHomePage))success
    fail:(nullable void (^)(void))fail;

+ (void)setScreenHomePageShowingClock:(BOOL)showing
    success:(nullable void (^)(BOOL showingClockOnHomePage))success
    fail:(nullable void (^)(void))fail;

```

3.66 Get/Set Homepage Style

Demo: QCSampleDisplayViewController

```

/**
 * @param currentStyle / style Common values: 1 / 2
 */
+ (void)getScreenHomePageStyleWithCurrentStyle:(NSInteger)currentStyle
    success:(nullable void (^)(NSInteger homePageStyle))success
    fail:(nullable void (^)(void))fail;

+ (void)setScreenHomePageStyle:(NSInteger)style
    success:(nullable void (^)(NSInteger homePageStyle))success
    fail:(nullable void (^)(void))fail;

```

3.67 Get/Set Screen-On Duration & Homepage Parameters

Demo: QCSampleDisplayViewController

```

/**
 * Get/set watch screen-on duration & homepage parameters.
 * @param opType 1=read, 2=write, 3=restore homepage picture to default
 * @param lightingSeconds Screen-on time in seconds (4~10)
 * @param homePageType 0=invalid, 1=steps, 2=calories, 3=weather, 4=heart rate
 * @param transparency Mask transparency 0~100
 * @param pictureType 0=default, 1=user-configured (read-only when writing)
 */
+ (void)setHomePageScreenOpType:(NSInteger)opType
    lightingSeconds:(NSInteger)lightingSeconds
    homePageType:(NSInteger)homePageType
    transparency:(NSInteger)transparency
    pictureType:(NSInteger)pictureType
    success:(nullable void (^)(NSInteger lightingSeconds, NSInteger homePageType, NSInteger
    transparency, NSInteger pictureType))suc
    fail:(nullable void (^)(void))fail;

/// Model-based read/write (recommended)
+ (void)setHomePageScreenOpType:(NSInteger)opType
    info:(nullable QCDimmingTimeInfo *)info
    success:(nullable void (^)(QCDimmingTimeInfo *))suc
    fail:(nullable void (^)(void))fail;

```

3.68 Get/Set Dial Index

Demo: QCSampleDisplayViewController

```
/**
 * Get dial display index. index: 0-N, 0=wallpaper.
 */
+ (void)getDialIndexWithFinished:(nullable void (^)(NSInteger index, NSError *_Nullable error))finished;

/**
 * Set dial display index (some devices only).
 */
+ (void)setDialIndexWith:(NSInteger)index finished:(nullable void (^)(NSError *_Nullable error))finished;
```

3.69 Get/Set Raise-to-Wake (Band)

Demo: QCSampleDisplayViewController (Palm / raise-to-wake)

Note: Use these APIs for band raise-to-wake. For ring devices, use the model APIs in 3.70; do not mix the two.

```
/**
 * Get band raise-to-wake info.
 * @param success isOn: enabled; flipType: wear hand, 1=left, 2=right
 */
+ (void)getFlipWristInfo:(nullable void (^)(BOOL isOn, NSUInteger flipType))success
    fail:(void (^)(void))fail;

/**
 * Set band raise-to-wake info.
 * @param on YES to enable / NO to disable
 * @param flipType 1=left, 2=right
 */
+ (void)setFlipWristOn:(BOOL)on
    flipType:(NSUInteger)flipType
    success:(nullable void (^)(BOOL featureOn, NSUInteger flipType))success
    fail:(nullable void (^)(void))fail;
```

3.70 Get/Set Flip-Wrist / Wear Info (Ring Model APIs)

Demo: QCSampleTouchGesturesViewController (FlipWrist / left-right wearing)

Note: Ring devices only (may include brightness and DND window fields). For band raise-to-wake, use 3.69.

```
/**
 * Get ring display/wear info (enable, left/right hand, brightness, DND window, etc.).
 */
+ (void)getFlipWristInfoFinished:(nullable void (^)(QCFlipWristInfoModel *_Nullable model, NSError *_Nullable error))finished;

/**
 * Set ring display/wear info.
 * @param model enable / flipType required; fill brightness, brightnessMax, timeMode, startTime, endTime per device capability
 */
+ (void)setFlipWristInfo:(QCFlipWristInfoModel *_Nullable)model
    finished:(nullable void (^)(NSError *_Nullable error))finished;
```

Key QCFlipWristInfoModel fields:

Field	Meaning
enable	Feature on/off
flipType	Wear hand: 1=left, 2=right
brightness / brightnessMax	Display brightness / max brightness (ring)
timeMode	1=custom DND window, other=off
startTime / endTime	DND start/end in minutes from midnight